

ISIT312 Big Data Management

Spark Stream Processing (DStreams)

Dr Guoxin Su and Dr Janusz R. Getta

School of Computing and Information Technology -
University of Wollongong

Spark Stream Processing (DStreams)

Outline

Spark Stream Processing Modules

Stream Processing

Sample Stream Processing

Spark Stream Processing Modules

Stream processing is a key requirement in many big data applications

As soon as an application computes something of value, for example, a report or a machine learning model, an organization may want to compute this result continuously in a production environment

This capability is lacked in **Hadoop MapReduce** framework due to slowness of hard-disk IO

In-memory computation implemented in **Spark** make stream processing possible

Spark Streaming based on its low-level API **Resilient Distributed Dataset** is available since Spark 1.2

Spark Structured Streaming based on the **Spark SQL** engine is available since Spark 2.1

Luckily, our VM has installation of Spark 3.5.1

Spark Stream Processing (DStreams)

Outline

[Spark Stream Processing Modules](#)

[Stream Processing](#)

[Sample Stream Processing](#)

Stream Processing

Stream processing is the act of continuously incorporating the new data in the stream to compute a result

Sample sources of streams

- Bank transactions
- Clicks on a website
- Sensor readings from IoT devices
- Scientific observations and experiments
- Manufacturing processes, and the others

Stream processing vs. batch processing

- Batch processing runs to a fixed set of data, but stream processing handles an unbounded set of data
- Batch processing has low timeliness requirement, but stream processing requires to work at near realtime

Stream Processing

Use cases of stream processing

- Notifications and alerting
- Real-time reporting
- Incremental ETL
- Update data to serve in real time
- Real-time decision making
- Online machine learning

Stream Processing

To see the challenges of stream processing, we consider the following example

Suppose we received the following data from a sensor

```
{value: 1, time: "2017-04-07T00:00:00"}  
{value: 2, time: "2017-04-07T01:00:00"}  
{value: 5, time: "2017-04-07T02:00:00"}  
{value: 10, time: "2017-04-07T01:30:00"}  
{value: 7, time: "2017-04-07T03:00:00"}
```

Sample data

What actions should be performed when receiving single values, say, 5 ?

How to react to a pattern, say, 2 -> 10 -> 5

What if data arrives out-of-order, for example, 10 before 5

Other issues: What if a machine in the system fails, losing some state?

What if the load is imbalanced? How can an application signal downstream consumers when analysis for some event is done, and so on

Stream Processing

Main challenges of stream processing are the following

- Processing out-of-order data based on application timestamps (also called event time)
- Maintaining large amounts of states
- Supporting high-data throughput
- Processing each event exactly once despite machine failures
- Handling load imbalance and stragglers
- Responding to events at low latency
- Joining with external data in other storage systems
- Determining how to update output sinks as new events arrive
- Writing data transactionally to output systems
- Updating application business logic at runtime

Spark Stream Processing (DStreams)

Outline

Spark Stream Processing Modules

Stream Processing

Sample Stream Processing

Sample Stream Processing

Stream Processing suppose to provide fast, scalable, fault-tolerant, end-to-end exactly-once stream processing without the user having to reason about streaming

Assume that we want to count the number of words in text data received from a data server listening on a TCP socket

Import `StreamingContext` and create a local `StreamingContext`

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext
sc = SparkContext("local[2]", "NetworkWordCount")
ssc = StreamingContext(sc, 1)
```

Import and create StreamingContext

Create a stream that represents streaming data from a TCP source

```
lines = ssc.socketTextStream("localhost", 9999)
```

Create a stream

Split the lines by space into words

```
words = lines.flatMap(lambda line: line.split(" "))
```

Create a stream

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

10/14

Sample Stream Processing

Count the words

```
pairs = words.map(lambda word: (word, 1))  
wordCounts = pairs.reduceByKey(lambda x, y: x + y)
```

Count

Print the first ten elements of each RDD to the console

```
wordCounts.pprint()
```

Print

Start the computations

```
ssc.start()
```

Start

Wait for the computation to terminate

```
ssc.awaitTermination()
```

Wait for termination

Finally, we save an application in a file `dstream.py`

Sample Stream Processing

The input is simulated by Netcat (a small utility found in most Unix-like systems) as a data server

```
nc -lk 9999
```

Starting Netcat

In a different Terminal, we start Spark-shell and input the Python code from the previous slides

Next, we input in the first Terminal window

```
nc -lk 9999  
aaa aaa aaa aa aa
```

Starting Netcat

Then we run the application in the second Terminal window

```
$SPARK_HOME/bin/spark-submit dstream.py localhost 9999
```

Running the application

Sample Stream Processing

Then we could see the following output in application Terminal window



```
-----  
Time: 2026-05-23 19:07:34  
-----  
  
-----  
Time: 2026-05-23 19:07:35  
-----  
( 'aaa', 3)  
( 'aa', 2)  
-----  
  
-----  
Time: 2026-05-23 19:07:36  
-----
```

References

[Spark Streaming Programming Guide](#)

[RDD Programming Guide](#)

Perrin J-G., Spark in Action, 2nd ed., Manning Publications Co. 2020

Chambers B., Zaharia M., Spark: The Definitive Guide, O'Reilly 2017